Informe del proyecto final

Universidad Mariano Gálvez

Curso: Análisis de Sistemas II

Proyecto final: Análisis, diseño arquitectónico, aplicación funcional, UX/UI y estrategia de calidad

Sistema: Biblioteca GG

Integrantes: Giovanni Francia y Gerardo Lucero

Docente: docente del curso

Fecha: 22 de octubre de 2026

1. Introducción y resumen ejecutivo

Biblioteca GG administra usuarios, el catálogo y los préstamos de una biblioteca. La fase anterior entregó un MVP en Node.js, Express y Sequelize con PostgreSQL, historias iniciales y escenarios Gherkin en inglés técnico. Esta entrega conserva ese dominio y lo completa: requerimientos formales, diseño UML e IEEE 1016, decisión de arquitectura, modelo C4, persistencia políglota, API ejecutable, contrato OpenAPI, errores RFC 9457, pruebas de aceptación, prototipo mobile-first y plan de calidad.

Los diez pasos quedan repartidos así. Los pasos 1 y 6 especifican y gestionan el trabajo. Los pasos 2, 4 y 5 deciden y dibujan la arquitectura y los datos. Los pasos 3 y 7 fijan el contrato y lo implementan con escenarios. Los pasos 8 y 9 evalúan el uso y la accesibilidad. El paso 10 cierra con el plan SQA, los defectos y la trazabilidad.

El resultado ejecutable responde autenticación, consulta con caché, alta de libro, detalle y alta de préstamo. La fuente relacional se confirma primero. La ficha documental y el grafo se replican después. Si no hay motores instalados, la misma lógica corre en memoria para que la demostración no dependa de cuatro servidores.

2. Requerimientos, modelado UML 2.5.1 y gestión ágil

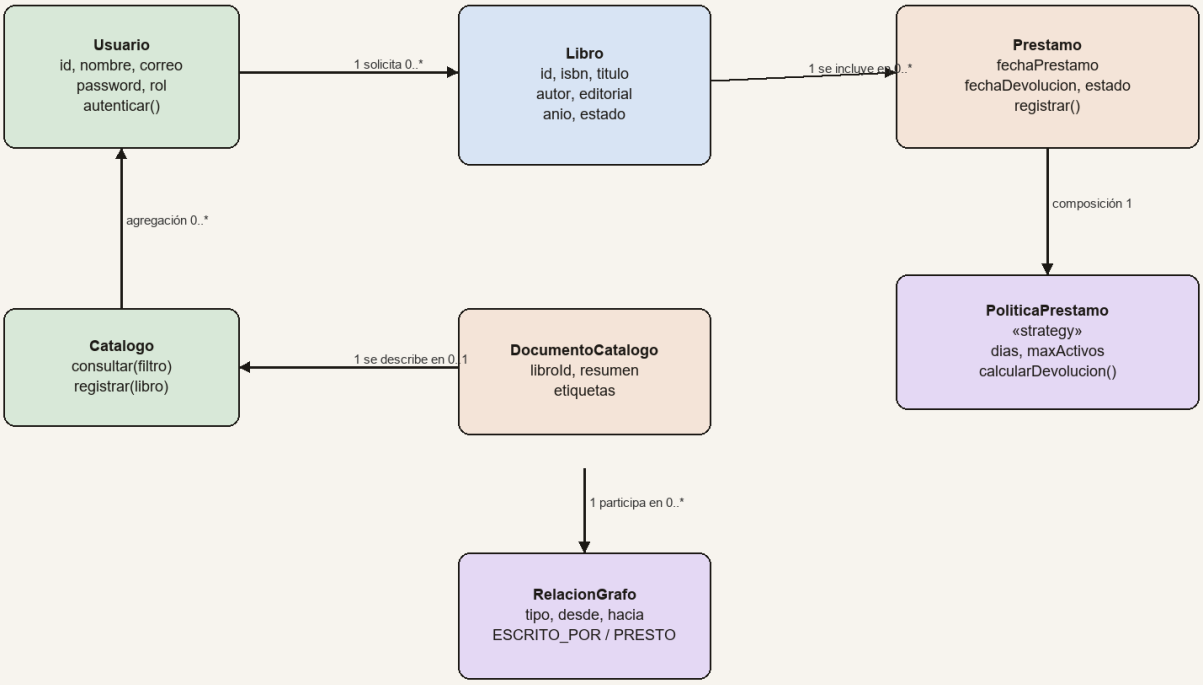
2.1 Especificación ISO/IEC/IEEE 29148:2018

Los requerimientos usan identificador único y la forma sujeto, verbo de obligación, acción y condición. El detalle está en docs/requirements/srs_ieee_29148.pdf. Los funcionales cubren autenticación, alta y consulta de libros, préstamo, política por rol, correo único y relación de grafo. Los no funcionales cubren Problem Details, TTL, autorización, confirmación relacional y la ausencia de la contraseña en las respuestas.

2.2 Diagramas UML 2.5.1

El diagrama de clases muestra composición entre el préstamo y su política, agregación entre el catálogo y los libros, y las cardinalidades de usuario, libro, ficha y grafo.

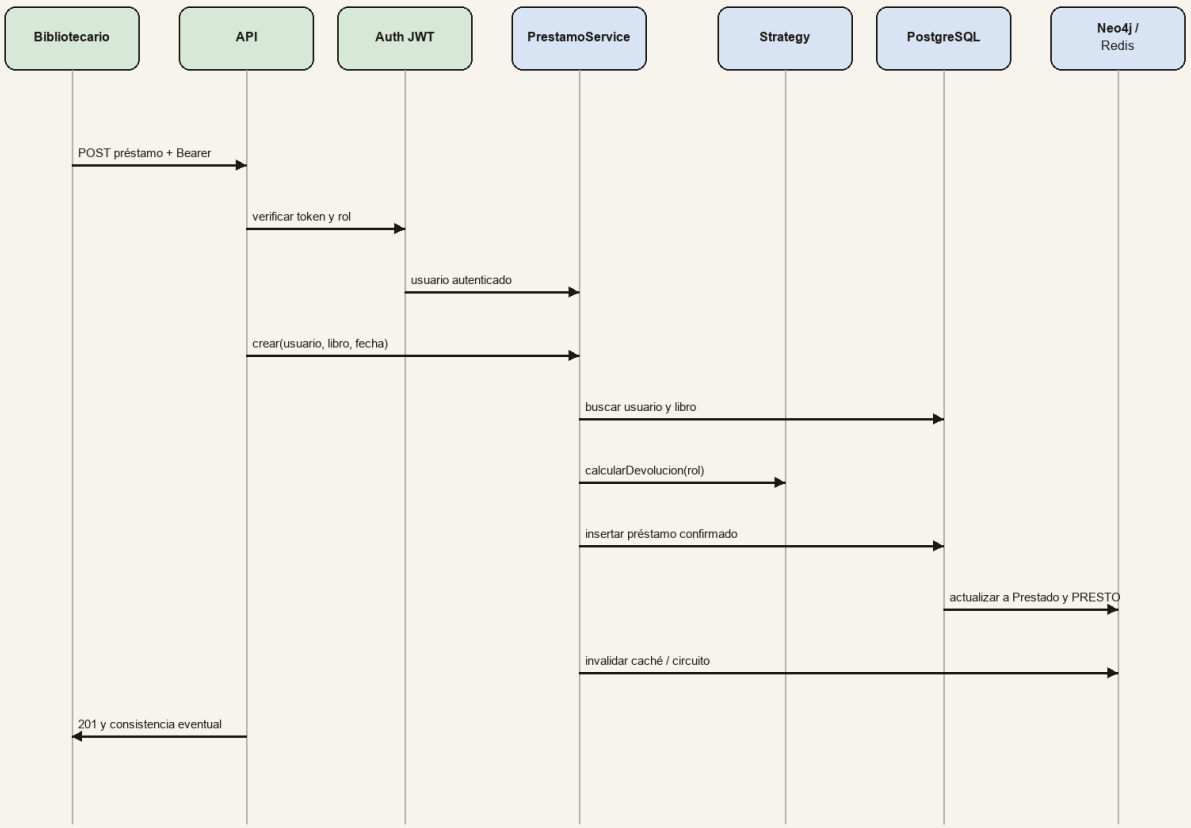
UML 2.5.1 · Diagrama de clases de análisis y diseño



Composición: el préstamo se calcula con una política. Agregación: el catálogo agrupa libros que también existen fuera de una consulta.

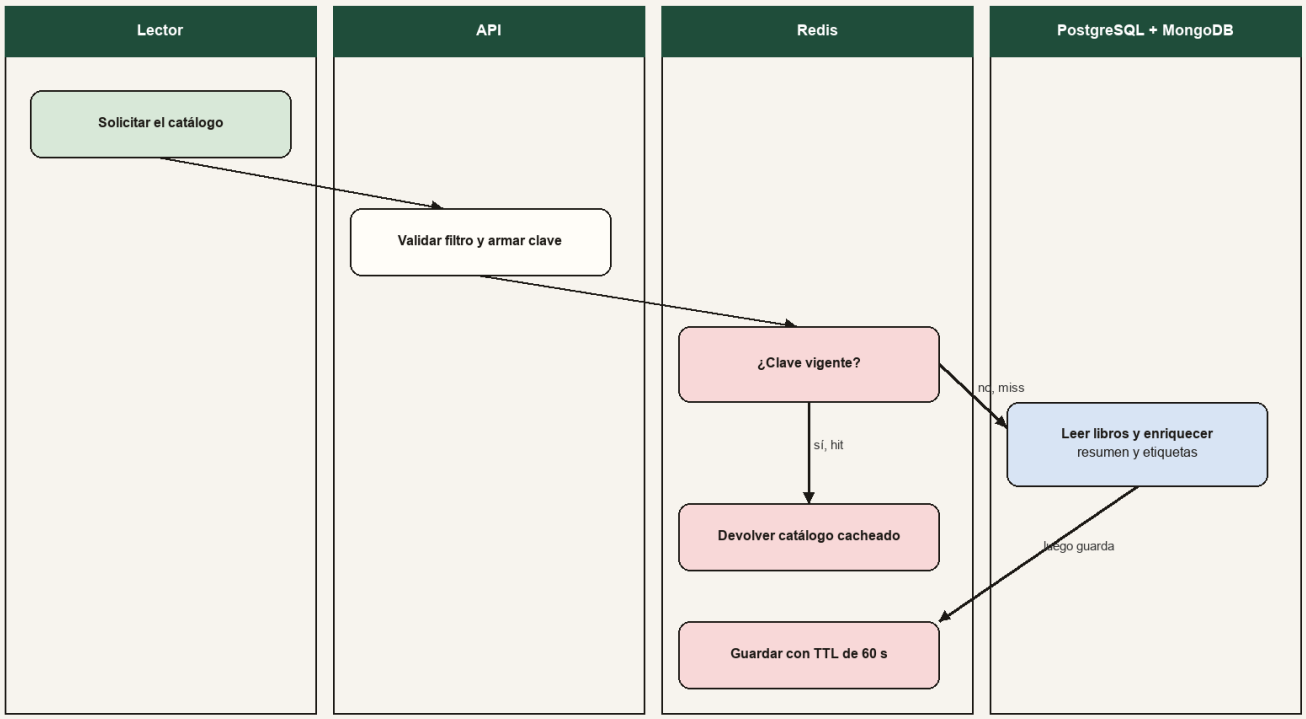
El diagrama de secuencia recorre el alta de un préstamo: token, política, confirmación relacional, observadores y caché.

UML 2.5.1 · Diagrama de secuencia del préstamo



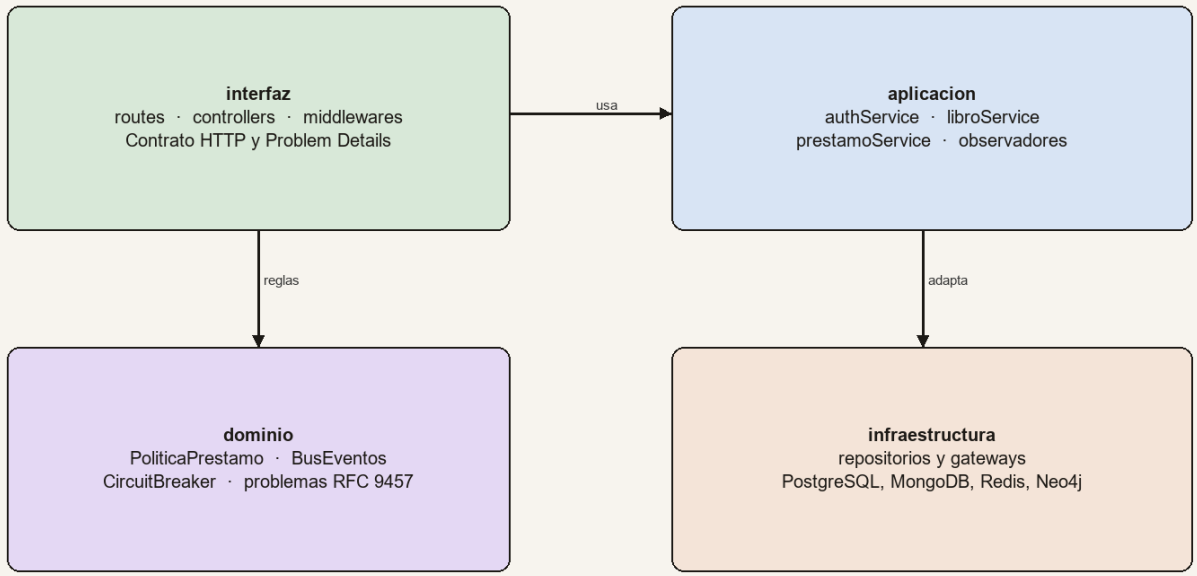
El diagrama de actividades usa calles para el lector, la API, Redis y la pareja PostgreSQL/MongoDB. El rombo es el acierto o el fallo de la caché.

UML 2.5.1 · Actividad con calles: consultar catálogo



El diagrama de paquetes separa interfaz, aplicación, dominio e infraestructura.

UML 2.5.1 · Diagrama de paquetes



Las fuentes editables están en docs/uml/*.puml.

2.3 Fichas IEEE 1016-2009

Las fichas de autenticación, catálogo, préstamos y errores describen atributos, interfaces, dependencias y excepciones. El documento es docs/requirements/fichas_diseno_ieee1016.pdf.

2.4 Gestión ágil

El marco es Scrum para el orden del trabajo y un tablero Kanban para ver el estado. El Product Owner prioriza y acepta según la Definición de Hecho; no reemplaza la prueba automática. Cada historia de docs/agile/backlog.md trae Card, Conversation y Confirmation, y se contrasta con INVEST. La Definición de Preparado exige identificador, criterios ejecutables y endpoint. La Definición de Hecho exige contrato, Problem Details cuando hay error, escenario Gherkin y prueba. El repositorio del tablero es https://github.com/gluceroa-sudo/BibliotecaGG.

3. Evaluación arquitectónica, C4 y persistencia

3.1 ISO/IEC 25010

Se compararon capas, repositorio, microservicios y eventos. Microservicios se rechazó por el costo de operación de un equipo de dos personas. Capas y repositorio se adoptaron. Los eventos se adoptaron dentro del proceso. Los escenarios de desempeño, fiabilidad, mantenibilidad y seguridad están en docs/architecture/escenarios_iso25010.pdf.

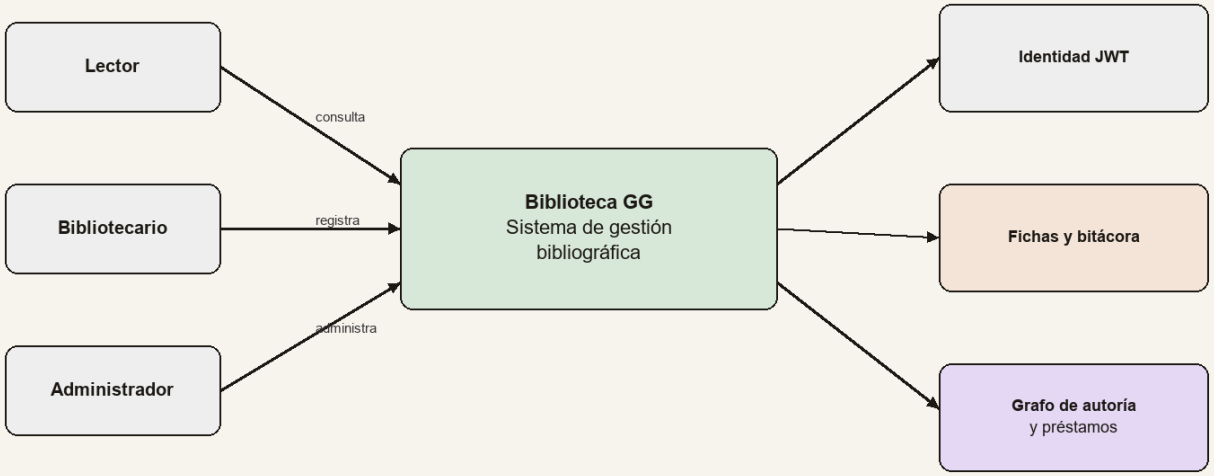
3.2 ADR e ISO/IEC/IEEE 42010

ADR-001 registra el estilo. ADR-002 registra la persistencia políglota. ADR-003 registra API-First y RFC 9457. ADR-004 registra Strategy, Repository, Gateway, Observer y Circuit Breaker. Cada registro tiene contexto, opción, consecuencias y estado aceptado.

3.3 Modelo C4

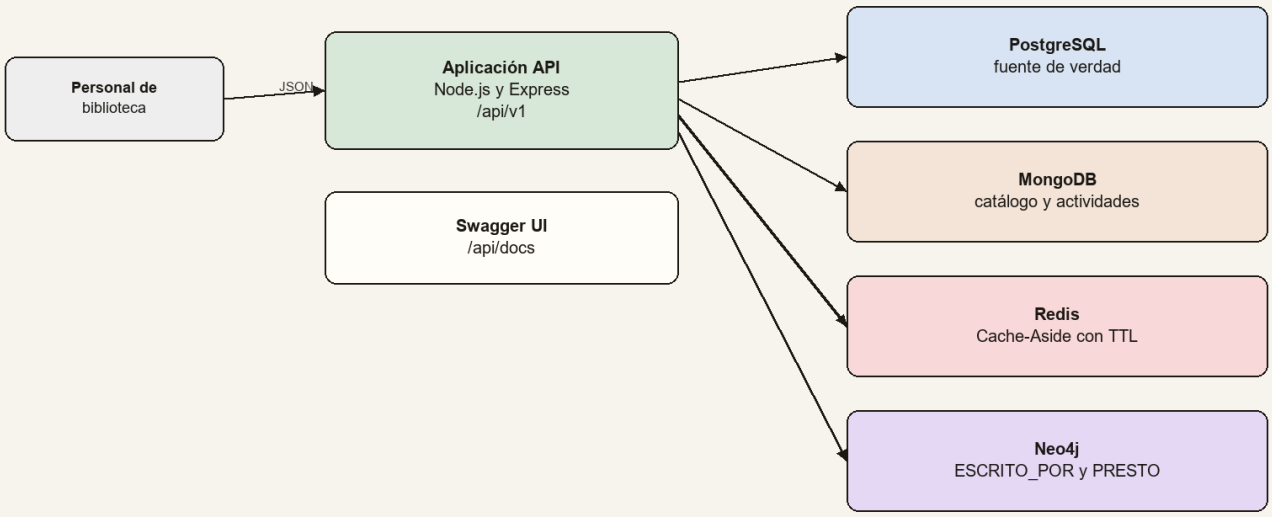
Nivel 1. El sistema, los actores y las dependencias de identidad, fichas y grafo.

C4 Nivel 1 · Contexto del sistema



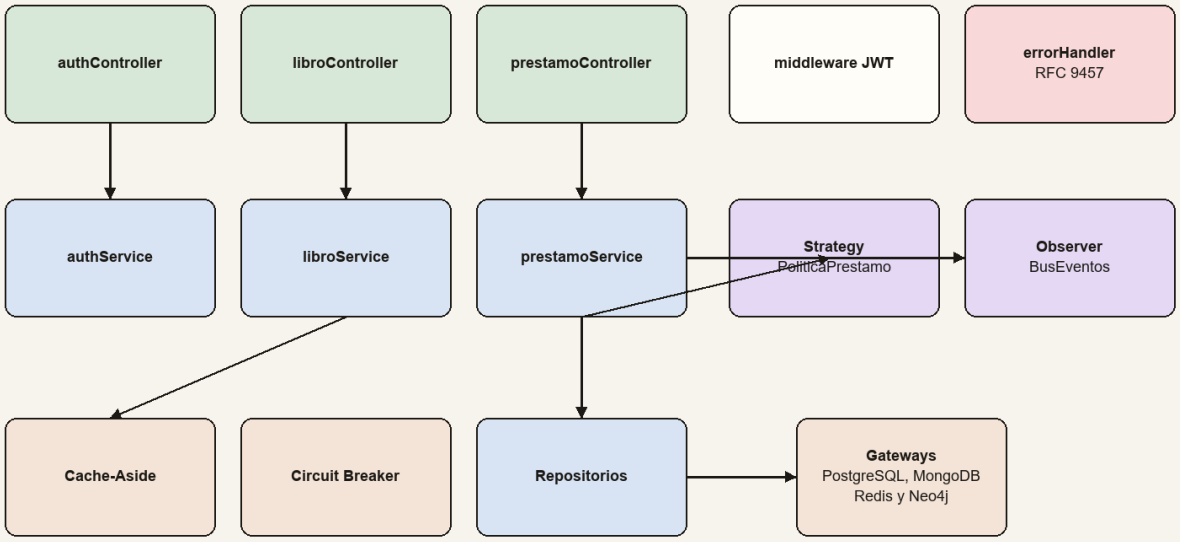
Nivel 2. Un contenedor de API, la documentación Swagger y cuatro almacenes.

C4 Nivel 2 · Contenedores



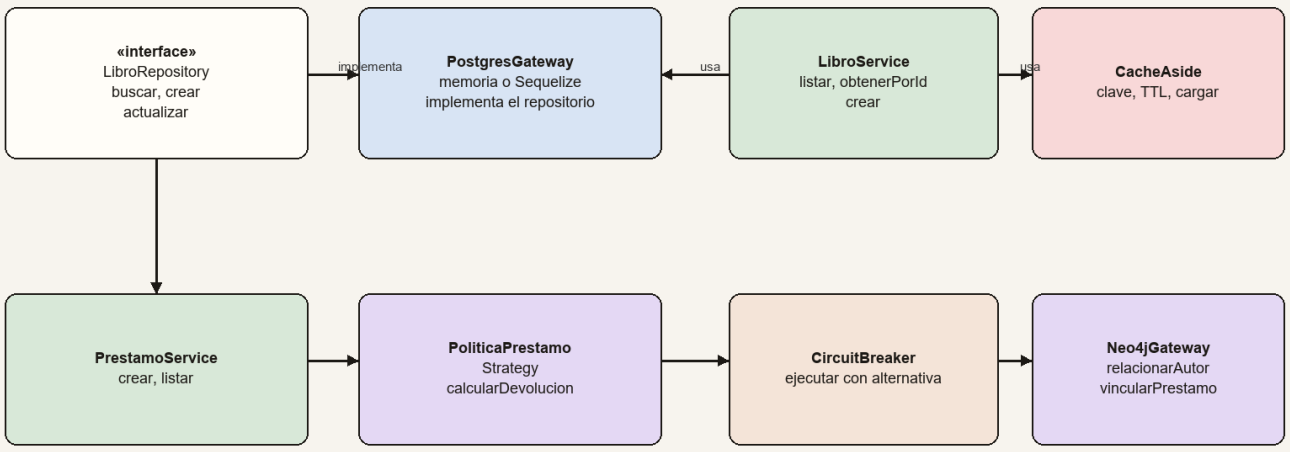
Nivel 3. Controladores, servicios, estrategia, observadores, caché, circuito, repositorios y gateways.

C4 Nivel 3 · Componentes del contenedor API



Nivel 4. La interfaz LibroRepository, el gateway relacional, Cache-Aside y el circuito hacia Neo4j.

C4 Nivel 4 · Código de los componentes críticos

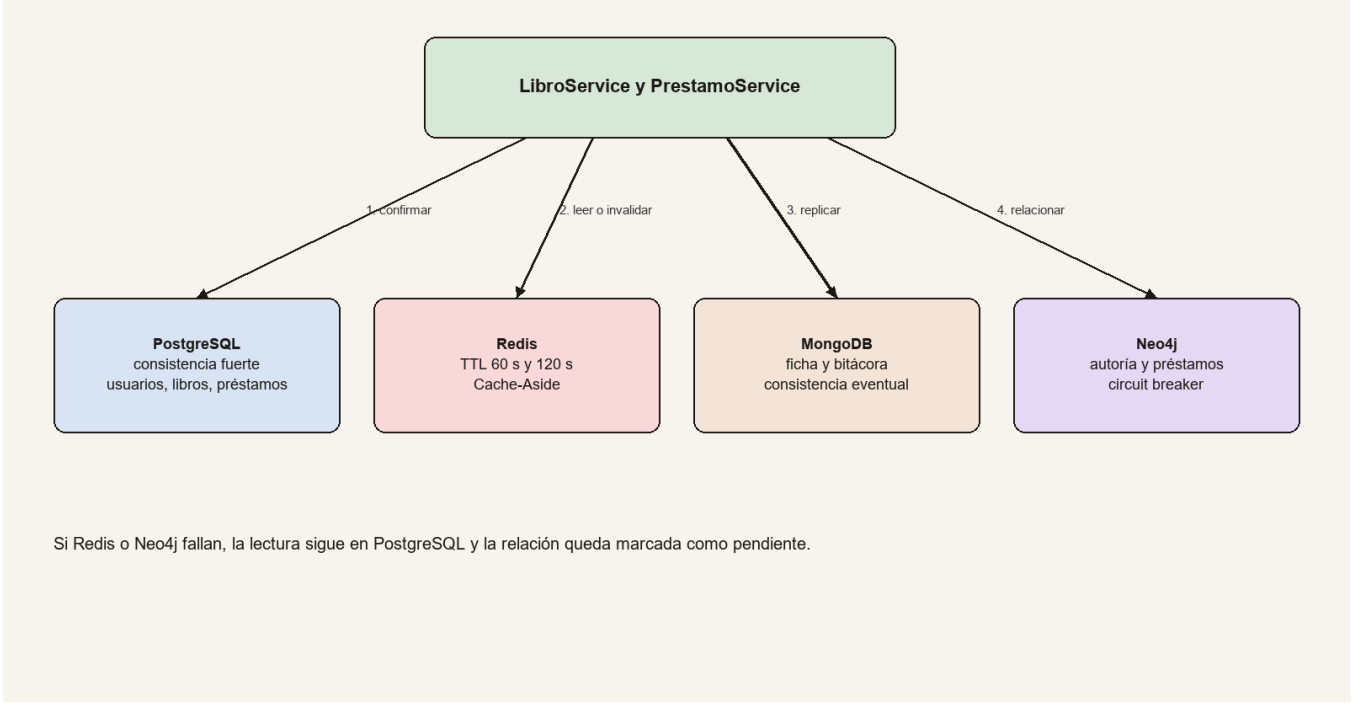


La interfaz aísla el caso de uso del motor. El circuito permite que un fallo del grafo no revierta el préstamo ya confirmado.

3.4 Persistencia políglota y Cache-Aside

PostgreSQL guarda usuarios, libros y préstamos porque esas reglas necesitan integridad. MongoDB guarda resumen, etiquetas y bitácora porque la ficha puede crecer sin alterar la tabla. Redis guarda listados 60 segundos y detalles 120 segundos. Neo4j guarda ESCRITO_POR y PRESTO. La lectura del catálogo es Cache-Aside: miss, carga, guarda; hit, responde. Una escritura invalida los prefijos. Si el grafo falla, el préstamo ya confirmado permanece y la actividad queda pendiente. Eso es consistencia eventual a propósito, no un olvido.

Persistencia políglota y patrón Cache-Aside



El script de creación está en docs/database/scripts_ddl.sql. Sin variables de conexión, los gateways usan memoria y los endpoints igual responden.

3.5 Patrones, cohesión y acoplamiento

Strategy aísla el reglamento. Repository aísla la colección. Gateway aísla cada motor. Observer reacciona al préstamo y al alta. Circuit Breaker impide que Redis o Neo4j tumben la operación ya decidida. La cohesión es por caso de uso. El acoplamiento de datos pasa por interfaces. El acoplamiento público es el YAML.

4. Diseño de APIs, implementación y BDD

4.1 API-First y OpenAPI 3.0.3

El contrato está en docs/api/openapi.yaml y se publica en /api/docs. Tiene esquemas, parámetros, seguridad Bearer y ejemplos. GraphQL sería una vista futura del grafo para preguntar, por ejemplo, qué libros relacionan a un autor con un lector, sin inventar un endpoint por cada recorrido. gRPC sería el transporte si el contenedor de préstamos se separa del de catálogo. Esta entrega no los implementa: deja el contrato HTTP que ambos podrían reutilizar. La nota está en

docs/api/integracion_graphql_grpc.md.

4.2 RFC 9110 y RFC 9457

Los códigos de estado son los de RFC 9110. El cuerpo de error es application/problem+json con type, title, status, detail e instance. La validación agrega errors. El éxito usa otro sobre, datos y meta, para que el cliente distinga un problema de un resultado.

4.3 Endpoints funcionales

Método	Endpoint	Descripción	Datos	Estado
POST	/api/v1/auth/login	Emite JWT	PostgreSQL	Funcional
GET	/api/v1/libros	Catálogo con caché	Redis y MongoDB	Funcional
POST	/api/v1/libros	Alta de libro	PostgreSQL y Neo4j	Funcional
GET	/api/v1/libros/{id}	Detalle	Redis y PostgreSQL	Funcional
POST	/api/v1/prestamos	Préstamo con política	PostgreSQL y Neo4j	Funcional

También responden GET /api/v1/usuarios, POST /api/v1/usuarios, GET /api/v1/prestamos y GET /api/v1/salud.

Cuentas de demostración: gerardo@bibliotecagg.edu con Biblioteca123!, y giovanni@bibliotecagg.edu con Admin123!.

4.4 BDD e ISO/IEC/IEEE 29119-3

features/autenticacion.feature y features/gestion_recursos.feature están en Gherkin con Dado, Cuando y Entonces, e incluyen Esquema del escenario y tablas de ejemplos. Los casos CP-01 a CP-08 están en docs/qa/casos_prueba_ieee29119.pdf. npm test ejecuta la comprobación automática.

5. Experiencia de usuario y mobile-first

5.1 ISO 9241-210 y recorrido

El reporte docs/ui/reporte_ux_ui.pdf describe el contexto de mostrador, el recorrido del bibliotecario y los riesgos de cada momento.

5.2 Heurísticas y WCAG 2.2

La inspección usa las diez heurísticas de Nielsen y los principios POUR en nivel AA: texto además de color, foco visible, etiquetas, destino de 44 píxeles y idioma declarado.

5.3 Atomic Design y prototipo

Átomos, moléculas y organismos arman las pantallas de acceso, catálogo, detalle y préstamo. El prototipo navegable es docs/ui/prototipo/index.html, pensado primero para el teléfono. Si el curso exige la URL de Figma, estas pantallas se publican allí y el enlace se agrega en el README; el flujo evaluado es el de este repositorio.

6. Estrategia de calidad

6.1 IEEE 730-2014 y shift-left

El plan docs/qa/plan_sqa_ieee730.pdf separa QA de QC. QA prepara requerimientos, contrato y criterios. QC ejecuta pruebas. El shift-left pone el escenario y la prueba junto al endpoint, no después de la demostración.

6.2 Pirámide

Abajo, política, circuito y vencimiento de caché. En el centro, la API con supertest. Arriba, Postman y el prototipo, en ejecución manual.

6.3 IEEE 1044-2009

El reporte docs/qa/reporte_defectos_ieee1044.pdf define severidad, prioridad y el ciclo reconocimiento, registro, análisis, resolución, verificación y cierre. Incluye un defecto de estado del libro ya cerrado y la devolución diferida, que no se presenta como función hecha.

7. Matriz de trazabilidad integral

Requerimiento	Historia	C4	Endpoint	Código	BDD	Caso
REQ-FN-001	HU-01	Auth	POST /api/v1/auth/login	authController.js	autenticacion.feature	CP-01
REQ-FN-002	HU-02	Catálogo	POST /api/v1/libros	libroService.js	gestion_recursos.feature	CP-02
REQ-FN-003	HU-03	Redis	GET /api/v1/libros	cacheAside.js	gestion_recursos.feature	CP-03
REQ-FN-005	HU-04	Préstamos	POST /api/v1/prestamos	prestamoService.js	gestion_recursos.feature	CP-05
REQ-FN-006	HU-05	Strategy	POST /api/v1/prestamos	strategy.js	gestion_recursos.feature	CP-06

La matriz completa, con no funcionales y REQ-FN-004, REQ-FN-007 y REQ-FN-008, está en docs/qa/trazabilidad.pdf.

8. Conclusiones

Escribir el requerimiento con una forma fija hizo visible qué historia estaba de más y cuál no tenía prueba. El contrato OpenAPI evitó que la colección Postman y el código inventaran cuerpos distintos. Los escenarios con tablas de ejemplos dejaron el ISBN duplicado y el libro inexistente al lado del caso feliz, no en un párrafo aparte.

El modelo C4 de cuatro niveles sirvió para decir lo mismo a tres distancias: quién usa el sistema, qué procesos y datos hay, qué componentes viven en la API y qué clases cierran el préstamo. Sin ese

corte, la persistencia políglota se ve como cuatro logos y no como una decisión con una fuente de verdad y tres réplicas.

La limitación consciente es la devolución del ejemplar y la publicación del prototipo en Figma. La limitación de operación es que la demostración por defecto usa memoria; el código de conexión y el DDL quedan listos para los motores reales.

9. Referencias

- ISO/IEC/IEEE 29148:2018. Requisitos del ciclo de vida.
- IEEE 1016-2009. Descripción del diseño de software.
- ISO/IEC 25010. Modelos de calidad.
- ISO/IEC/IEEE 42010:2011. Descripción de la arquitectura.
- Simon Brown. El modelo C4.
- OpenAPI Specification 3.0.3.
- RFC 9110. Semántica HTTP.
- RFC 9457. Problem Details.
- ISO/IEC/IEEE 29119-3:2021. Documentación de pruebas.
- ISO 9241-210 e ISO 9241-11. Diseño centrado en el usuario y usabilidad.
- W3C. WCAG 2.2.
- IEEE 730-2014. Planes de aseguramiento de calidad.
- IEEE 1044-2009. Clasificación de anomalías.
- Repositorio de la fase previa y de esta entrega: https://github.com/gluceroa-sudo/BibliotecaGG